



{ }

C++

GHI CHÚ

- Hướng Dẫn Nhập Môn C++ -



CODE

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, C++!";
    return 0;
}
```



SUY NGHĨ



HỌC



GIẢI

> NỘI DUNG <

- | | |
|-----------------------|-------------------------|
| ▶ Giới thiệu về C++ | ▶ Khái Niệm OOP |
| ▶ Biến & Kiểu Dữ Liệu | ▶ Kế Thừa (Inheritance) |
| ▶ Toán Tử | ▶ Templates |
| ▶ Input / Output | ▶ STL |
| ▶ Câu Lệnh Điều Khiển | ▶ Xử Lý File |
| ▶ Hàm (Functions) | ▶ Modern C++ |
| ▶ Mảng & Chuỗi | ▶ C++ Nâng Cao |
| ▶ Con Trỏ (Pointers) | ▶ Và Nhiều Hơn Nữa... |

"Viết code. Biên dịch thành công. Debug kiên nhẫn. Lặp lại."



» C++ CƠ BẢN «

— Cú pháp, cấu trúc, biên dịch, biến, kiểu dữ liệu, toán tử —

1. C++ LÀ GÌ ?

C++ là ngôn ngữ lập trình bậc cao, đa mục đích. Đây là phần mở rộng của ngôn ngữ C. Nó hỗ trợ cả lập trình thủ tục (procedural) và lập trình hướng đối tượng (object-oriented).

C++ = C + OOP

C++

2. ĐẶC ĐIỂM CỦA C++

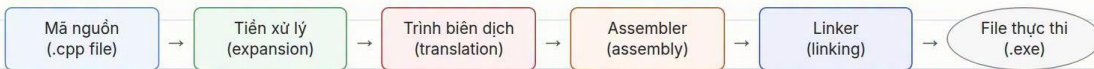
- Đơn giản và hiệu quả
- Ngôn ngữ bậc trung
- Portable (đa nền tảng)
- Hỗ trợ OOP
- Thư viện chuẩn phong phú (STL)
- Nhanh và được dùng rộng rãi

3. CẤU TRÚC CHƯƠNG TRÌNH C++

```
#include <iostream>    // Chỉ thị tiền xử lý
using namespace std;  // Namespace

int main() {           // hàm main
    cout << "Hello, C++!"; // câu lệnh
    return 0;          // return
}                      // kết thúc main
```

4. QUÁ TRÌNH BIÊN DỊCH



Bước: Mã nguồn → Tiền xử lý → Trình biên dịch → Assembler → Linker → File thực thi

5. BIẾN (VARIABLES)

Biến (variable) là một **vùng nhớ có tên** dùng để lưu trữ dữ liệu. Giá trị của nó có thể thay đổi trong quá trình chương trình chạy.

Cú pháp:

kiểu_dữ_liệu tên_biến = giá_trị;

Ví dụ:

int age = 20;

QUY TẮC ĐẶT TÊN BIẾN

1. Tên biến phải bắt đầu bằng chữ cái hoặc dấu gạch dưới (_).
2. Có thể chứa chữ cái, chữ số và dấu gạch dưới.
3. Không được chứa khoảng trắng hoặc ký tự đặc biệt.
4. Phân biệt chữ hoa/thường (age, Age và AGE là khác nhau).
5. Không được trùng với từ khóa (keyword).

6. KIỂU DỮ LIỆU TRONG C++

Kiểu dữ liệu	Kích thước (byte)	Ví dụ	Mô tả
int	4	int a = 10;	Lưu giá trị số nguyên
float	4	float f = 3.14;	Lưu giá trị số thực
double	8	double d = 3.14159;	Lưu giá trị số thực độ chính xác kép
char	1	char c = 'A';	Lưu một ký tự
bool	1	bool b = true;	Lưu giá trị đúng/sai



» C++ INPUT & OUTPUT «



codedsoul_05



cin , cout , getline , formatting

1. INPUT & OUTPUT CƠ BẢN

- `cin` → dùng để nhận input từ người dùng.
- `cout` → dùng để hiển thị output ra màn hình.

INPUT (cin)	OUTPUT (cout)
<code>int a;</code>	<code>int a = 10;</code>
<code>cin >> a; // nhập số nguyên</code>	<code>cout << a; // 10</code>
<code>float f;</code>	<code>float f = 2.5;</code>
<code>cin >> f; // nhập số thực</code>	<code>cout << f; // 2.5</code>
<code>char ch;</code>	<code>char ch = 'A';</code>
<code>cin >> ch; // nhập ký tự</code>	<code>cout << ch; // A</code>
<code>string s;</code>	<code>string s = "Hello";</code>
<code>cin >> s; // nhập 1 từ</code>	<code>cout << s; // Hello</code>

2. HÀM GETLINE()

- `getline()` dùng để đọc toàn bộ 1 dòng, kể cả khoảng trắng.
- Được khai báo trong header `<string>`.

Cú pháp:

```
getline(cin, tên_biến);  
getline(cin, tên_biến, ký_tự_kết_thúc);
```

Ví dụ:

```
string name, line;  
cout << "Nhập tên: ";  
getline(cin, name); // đọc trọn dòng  
cout << "Nhập một câu: ";  
getline(cin, line); // đọc trọn dòng  
cout << "Tên: " << name << endl;  
cout << "Câu: " << line << endl;
```

3. KẾT HỢP cin >> VÀ getline()

Vấn đề: Sau khi dùng `cin >>`, ký tự xuống dòng (`\n`) vẫn còn trong buffer. Vì vậy `getline()` có thể đọc phải dòng trống.

Giải pháp: Dùng `cin.ignore()` để xóa buffer.

```
int age;  
cin >> age;  
cin.ignore(); // xóa ký tự xuống dòng còn sót  
string name;  
getline(cin, name); // giờ đã hoạt động đúng
```

4. ĐỊNH DẠNG OUTPUT VỚI cout

Manipulator	Header	Mô tả	Output
<code>endl</code>	<code><iostream></code>	Xuống dòng & flush buffer	Hi
<code>setw(n)</code>	<code><iomanip></code>	Đặt độ rộng field (tối thiểu)	42
<code>setprecision(n)</code>	<code><iomanip></code>	Đặt số chữ số thập phân	3.14
<code>fixed</code>	<code><iomanip></code>	Hiển thị dạng fixed-point	3.14
<code>left</code>	<code><iomanip></code>	Căn trái trong độ rộng	Hi
<code>right</code>	<code><iomanip></code>	Căn phải trong độ rộng	Hi

ĐIỂM CHÍNH CẦN NHỚ

- ★ `cin >>` đọc input đến khi gặp khoảng trắng đầu tiên.
- ★ `getline()` đọc toàn bộ dòng, kể cả khoảng trắng.
- ★ Dùng `cin.ignore()` sau `cin >>` trước khi `getline()`.
- ★ Dùng manipulator để định dạng output gọn gàng, dễ đọc.

5. VÍ DỤ ĐẦY ĐỦ

```
#include <iostream>  
#include <string>  
#include <iomanip>  
using namespace std;  
  
int main() {  
    int age;  
    string name;  
    cout << "Nhập tuổi: ";  
    cin >> age;  
    cin.ignore(); // xóa buffer  
    cout << "Nhập tên: ";  
    getline(cin, name);  
  
    cout << "\n--- Kết quả ---\n";  
    cout << "Tuổi: " << setw(3) << age << endl;  
    cout << "Tên: " << left << setw(10) << name << endl;  
  
    cout << fixed << setprecision(2);  
    cout << "Giá trị PI: " << 3.14159 << endl;  
    return 0;  
}
```

Chạy &
Luyện tập!

"Chất lượng output phản ánh chất lượng code của bạn." ✨

ĐÚNG

SAI

» ĐIỀU KHIỂN LƯỒNG C++ «



★

Câu lệnh điều khiển luồng quyết định thứ tự thực thi các câu lệnh trong chương trình.

1. IF - ELSE

Dùng để thực thi 1 khối lệnh nếu điều kiện đúng hoặc sai.

```
if (condition) {
    // code nếu đúng
} else {
    // code nếu sai
}
```

Điều kiện?

Đúng ↙ ↘ Sai

khối if

khối else

Ví dụ:

```
int age = 18;
if (age >= 18) {
    cout << "Bạn đủ tuổi bầu cử.";
} else {
    cout << "Bạn chưa đủ tuổi bầu cử.";
}
```

2. SWITCH CASE

Dùng khi có nhiều lựa chọn. Đây là cách gọn hơn so với dùng nhiều if-else.

```
switch (expression) {
    case value1:
        // code;
        break;
    case value2:
        // code;
        break;
    default:
        // code;
}
```

Ví dụ:

```
int day = 2;
switch (day) {
    case 1: cout << "Thứ Hai"; break;
    case 2: cout << "Thứ Ba"; break;
    default: cout << "Ngày không hợp lệ";
}
```

3. VÒNG LẶP (LOOPS) Dùng để thực thi 1 khối lệnh nhiều lần.

(a) for loop

```
for (khởi_tạo; điều_kiện;
cập_nhật){
    // code cần thực thi
}
```

Ví dụ:

```
for (int i=1; i<=5; i++)
{
    cout << i << " ";
}
```

Output: 1 2 3 4 5

(b) while loop

```
while (điều_kiện) {
    // code
    // cập nhật
}
```

Ví dụ:

```
int i = 1;
while (i <= 5) {
    cout << i << " "; i++;
}
```

Output: 1 2 3 4 5

(c) do-while loop

```
do {
    // code
    // cập nhật
} while (điều_kiện);
```

Ví dụ:

```
int i = 1;
do {
    cout << i << " "; i++;
} while (i <= 5);
```

Output: 1 2 3 4 5

Khi nào dừng?

- for loop — Khi biết trước số lần lặp.

- while loop — Khi chưa biết trước số lần lặp.

- do-while — Luôn thực thi ít nhất 1 lần.

4. CÂU LỆNH BREAK

Dùng để dừng vòng lặp hoặc switch ngay lập tức.

Trong vòng lặp:

```
for (int i=1; i<=10; i++) {
    if (i == 5) break; // dừng ở 5
    cout << i << " ";
}
```

Output: 1 2 3 4

5. CÂU LỆNH CONTINUE

Dùng để bỏ qua lần lặp hiện tại và chuyển sang lần tiếp theo.

```
for (int i=1; i<=5; i++) {
    if (i == 3) continue; // bỏ qua khi i=3
    cout << i << " ";
}
```

Output: 1 2 4 5

ĐIỂM CHÍNH CẦN NHỚ ★ if-else → quyết định dựa trên điều kiện ★ switch → nhiều lựa chọn



» MẢNG & CHUỖI TRONG C++ «



Mảng và chuỗi dùng để lưu nhiều giá trị cùng kiểu dữ liệu dưới một tên duy nhất.

1. MẢNG 1 CHIỀU (1D ARRAYS)

Mảng 1D lưu các phần tử trên một hàng duy nhất.

kiểu_dữ_liệu
tên_mảng[kích_thuớc];

Biểu diễn bộ nhớ:

0	1	2	3	4
10	20	30	40	50

Ví dụ:

```
int arr[5] = {10, 20, 30, 40, 50};
cout << arr[0]; // 10
cout << arr[4]; // 50
```

Thao tác thường dùng

- Duyệt mảng (Traversal)
- Chèn (Insertion)
- Xóa (Deletion)
- Tìm kiếm (Searching)

2. MẢNG 2 CHIỀU (2D ARRAYS)

Mảng 2D lưu các phần tử dưới dạng hàng và cột.

kiểu_dữ_liệu tên_mảng[hàng][cột];

Ví dụ:

```
int mat[2][3] = { {1, 2, 3},
                  {4, 5, 6} };
cout << mat[0][1]; // 2
cout << mat[1][2]; // 6
```

Biểu diễn bộ nhớ:

	Cột 0	Cột 1	Cột 2
Hàng 0	1	2	3
Hàng 1	4	5	6

Lưu ý:

- Đánh chỉ số bắt đầu từ 0.
- Tổng số phần tử = hàng × cột

3. CHUỖI KIỂU C (MẢNG KÝ TỰ)

Chuỗi kiểu C là một mảng ký tự kết thúc bằng ký tự null '\0'.

Khai báo:

```
char str[10];
```

Khởi tạo:

```
char str[] = "Hello";
```

Ví dụ:

```
char str[20] = "Hello, World!";
cout << str; // Hello, World!
cout << strlen(str); // 13
```

Hoạt động như thế nào?

H	e	l	l	o	\0
0	1	2	3	4	5

Hàm quan trọng (trong <cstring>)

strlen(str) → độ dài chuỗi (không tính '\0')
strcpy(dest, src) → copy src vào dest
strcat(dest, src) → nối src vào cuối dest
strcmp(s1, s2) → so sánh chuỗi (trả về 0 nếu bằng nhau)

Chương trình ví dụ:

```
char str1[20] = "Code";
char str2[20] = "Soul";
strcat(str1, str2); // str1 = CodeSoul
cout << str1; // CodeSoul
cout << strcmp("abc", "abc"); // 0
cout << strcmp("abc", "abd"); // < 0
```

4. std::string (CHUỖI C++)

std::string là một class trong thư viện chuẩn C++ giúp xử lý chuỗi dễ dàng và an toàn hơn.

Khai báo:

```
string str;
```

Khởi tạo:

```
string str = "Hello";
```

Ví dụ:

```
string s = "Hello, C++!";
cout << s; // Hello, C++!
cout << s.length(); // 10
cout << s[0]; // H
s = s + " World"; // Nối chuỗi
cout << s; // Hello, C++! World
```

Các hàm thành viên hữu ích

- s.length() → trả về độ dài
- s.empty() → kiểm tra rỗng
- s.append("text") → thêm vào cuối
- s.insert(pos, "text") → chèn tại vị trí
- s.erase(pos, len) → xóa ký tự
- s.substr(pos, len) → trả về chuỗi con

5. SO SÁNH NHANH

Đặc điểm	Chuỗi kiểu C (mảng ký tự)	std::string (chuỗi C++)
Header	<cstring>	<string>



```
int *p
&a → địa chỉ
*p → giá trị
```

» CON TRỎ & THAM CHIẾU C++ «

Con trỏ (pointer) lưu địa chỉ của biến. Tham chiếu (reference) là một tên khác (alias) cho một biến. Chúng giúp sử dụng bộ nhớ và đối số hàm hiệu quả.

1. CON TRỎ (POINTERS)

- Con trỏ là biến lưu địa chỉ của một biến khác.

Khai báo:

```
kiểu_dữ_liệu *tên_con_trỏ;
```

Ví dụ:

```
int a = 10;
int *p;    // con trỏ trỏ đến int
p = &a;    // lưu địa chỉ của a
cout << a << " " << *p << endl; // 10 10
cout << &a << " " << p << endl; // cùng địa chỉ
```

2. THAM CHIẾU (REFERENCES)

- Tham chiếu là một tên khác (alias) cho biến đã tồn tại.
- Phải khởi tạo ngay khi khai báo.

Khai báo:

```
kiểu_dữ_liệu &tên_tham_chiếu = biến;
```

Ví dụ:

```
int a = 20;
int &r = a; // r là tham chiếu đến a
r = 30;    // thay đổi giá trị a
cout << a << " " << r << endl; // 30 30
```

Điểm chính

- ★ Không thể là NULL
- ★ Không thể thay đổi để tham chiếu biến khác
- ★ Không tốn thêm bộ nhớ

3. SỐ HỌC CON TRỎ (POINTER ARITHMETIC)

- Có thể thực hiện các phép toán số học trên con trỏ.
- Con trỏ di chuyển theo bước bằng kích thước kiểu dữ liệu.

Ví dụ:

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // trỏ đến arr[0]
cout << *p << " "; // 10
p++;
cout << *p << " "; // 20
p = p + 2;
cout << *p << " "; // 40
cout << p - arr << endl; // 3 (chỉ số thứ 3)
```

arr

10	20	30	40	50
[0]	[1]	[2]	[3]	[4]

p ↑

p++ ↑

p=p+2 ↑

Phép toán	Ý nghĩa
p + n	di chuyển tới n phần tử phía sau
p - n	di chuyển lùi n phần tử
p++	tới phần tử tiếp theo
p--	về phần tử trước
p2 - p1	số phần tử giữa p1, p2

4. CẤP PHÁT BỘ NHỚ ĐỘNG

- Dùng khi không biết kích thước lúc biên dịch.
- Bộ nhớ được cấp phát lúc chạy bằng new và giải phóng bằng delete.

(a) Toán tử new

```
kiểu_dữ_liệu *ptr = new kiểu_dữ_liệu; // 1 biến
kiểu_dữ_liệu *arr = new kiểu_dữ_liệu[size]; // mảng
```

Ví dụ:

```
int *p = new int;
*p = 100;
int n;
cout << "Nhập kích thước: "; cin >> n;
int *arr = new int[n];
for(int i=0; i<n; i++) cin >> arr[i];
```

(b) Toán tử delete

```
delete ptr; // cho 1 biến
delete[] arr; // cho mảng
```

Ví dụ:

```
delete p;
p = nullptr; // cách làm tốt

delete[] arr;
arr = nullptr;
```

Lưu ý:

- ★ Luôn dùng delete để giải phóng bộ nhớ, tránh rò rỉ bộ nhớ.
- ★ Dùng nullptr sau khi delete.



» HÀM TRONG C++ «



Hàm (function) là một khối lệnh thực hiện một tác vụ cụ thể.
Nó giúp tổ chức code, tái sử dụng và dễ hiểu hơn.

1. KHAI BÁO HÀM

Cho trình biên dịch biết tên hàm, kiểu trả về và tham số.

Cú pháp:

```
kiểu_trả_về tên_hàm ( danh_sách_tham_số );
```

Ví dụ (Khai báo):

```
int add(int a, int b); // khai báo hàm
```

2. ĐỊNH NGHĨA HÀM

Phần thân thực tế của hàm, nơi tác vụ được thực hiện.

```
int add(int a, int b) { // định nghĩa
    int sum = a + b;    // logic
    return sum;         // giá trị trả về
}
```

Ví dụ (Gọi hàm):

```
int result = add(5, 3); // gọi hàm
```

3. THAM SỐ (PARAMETERS)

Tham số là các biến được liệt kê trong dấu ngoặc của định nghĩa hàm.

Các loại tham số

- Tham số thực (Actual Parameters) - dùng khi gọi hàm
- Tham số hình thức (Formal Parameters) - dùng trong định nghĩa hàm

add(5, 3)

5 3 (Tham số thực)

int add(int a, int b) { ... }

a b (Tham số hình thức)

4. GIÁ TRỊ TRẢ VỀ

Một hàm có thể trả về giá trị cho nơi gọi bằng câu lệnh return.

Cú pháp:

```
return giá_trị;
```

Ví dụ:

```
int square(int n) {
    return n * n;
}
```

💡 Nếu kiểu trả về là void, hàm không trả về giá trị nào.

5. TRUYỀN THEO GIÁ TRỊ (PASS BY VALUE)

Một bản sao của giá trị thực được truyền vào hàm. Thay đổi trong hàm KHÔNG ảnh hưởng đến giá trị gốc.

Ví dụ:

```
void changeValue(int x) {
    x = x + 10; // chỉ đổi bản sao
}

int main() {
    int a = 5;
    changeValue(a);
    cout << a; // Output: 5
}
```

"a" vẫn là 5 (không đổi)

6. TRUYỀN THEO THAM CHIẾU (PASS BY REFERENCE)

Địa chỉ của giá trị thực được truyền bằng tham chiếu (&). Thay đổi trong hàm ảnh hưởng đến giá trị gốc.

Ví dụ:

```
void changeValue(int &x) {
    x = x + 10; // đổi giá trị gốc
}

int main() {
    int a = 5;
    changeValue(a);
    cout << a; // Output: 15
}
```

"a" trở thành 15 (đã đổi)

7. VÍ DỤ ĐẦY ĐỦ

```
#include <iostream>
using namespace std;
// Khai báo hàm
int multiply(int a, int b);
```

```
// Định nghĩa hàm
int multiply(int a, int b) {
    int product = a * b;
    return product;
}
```

ĐIỂM CHÍNH CẦN NHỚ ✓ Hàm giúp tái sử dụng code ✓ Dùng tham số để truyền dữ liệu



» STRUCT, UNION & ENUM TRONG C++ «

codedsoul_05



Struct, Union và Enum là các kiểu dữ liệu do người dùng định nghĩa, giúp nhóm các dữ liệu liên quan lại với nhau.

1. STRUCT (Cấu trúc)

- Dùng để nhóm các kiểu dữ liệu khác nhau dưới một tên.

Cú pháp:

```
struct tên_struct {  
    kiểu_dữ_liệu thành_viên1;  
    kiểu_dữ_liệu thành_viên2;  
    ...  
};
```

Ví dụ:

```
struct Student {  
    int roll;  
    string name;  
    float marks;  
};  
Student s1 = {1, "Nikhil", 95.5};  
cout << s1.name << " " << s1.marks;
```

Các thành viên của struct có thể khác kiểu dữ liệu.

2. UNION

- Giống struct nhưng tất cả thành viên dùng chung 1 vùng nhớ.
- Chỉ một thành viên giữ giá trị tại một thời điểm.

Cú pháp:

```
union tên_union {  
    kiểu_dữ_liệu thành_viên1;  
    kiểu_dữ_liệu thành_viên2;  
    ...  
};
```

Ví dụ:

```
union Data {  
    int i; float f; char ch;  
};  
Data d;  
d.i = 10;  
cout << d.i; // 10  
d.f = 3.14;  
cout << d.f; // 3.14  
cout << d.i; // rác (chung vùng nhớ)
```

Tất cả thành viên bắt đầu từ cùng một vị trí bộ nhớ.

3. ENUM (Kiểu liệt kê)

- Dùng để định nghĩa tập các hằng số nguyên có tên.

Cú pháp:

```
enum tên_enum {  
    hằng1, hằng2, ...  
};
```

Ví dụ:

```
enum Day { SUN, MON, TUE, WED,  
          THU, FRI, SAT };  
Day d = MON;  
cout << d; // 1 (MON)  
cout << THU; // 4  
enum Day d2 = FRI;  
if(d2==FRI) cout<<"Friday!";
```

Mặc định, hằng đầu tiên = 0
SUN=0, MON=1, TUE=2, ...

4. TYPEDEF

- Dùng để tạo bí danh (tên mới) cho kiểu dữ liệu đã có.
- Cải thiện khả năng đọc code.

```
typedef kiểu_cũ tên_mới;  
typedef struct tên_struct tên_mới;
```

Ví dụ:

```
typedef unsigned long long ull;  
ull bigNumber = 1234567890123ULL;  
  
struct Point { int x, y; };  
typedef struct Point Point; // dùng 'Point'  
Point p1 = {10, 20};
```

5. USING (C++11 trở lên)

- Lựa chọn hiện đại thay cho typedef.
- Đễ đọc hơn và được ưa chuộng trong C++ hiện đại.

```
using tên_mới = kiểu_cũ;  
using tên_mới = struct tên_struct;
```

Ví dụ:

```
using ull = unsigned long long;  
ull bigNumber = 1234567890123ULL;  
  
struct Point { int x, y; };  
using Point = Point;  
Point p2 = {30, 40};
```

6. SO SÁNH

Đặc điểm	struct	union	enum	typedef	using
Mục đích	Nhóm các kiểu dữ liệu khác nhau	Chia sẻ bộ nhớ giữa các thành viên	Định nghĩa hằng số có tên	Tạo bí danh (cách cũ)	Tạo bí danh (cách mới)
Bộ nhớ	Mỗi thành viên có bộ nhớ riêng	Tất cả thành viên dùng chung bộ nhớ	Giống int (kích thước int)	Giống kiểu gốc	Giống kiểu gốc
Khởi tạo	Có thể khởi tạo tất cả thành viên	Chỉ một thành viên tại một thời điểm	Mặc định là hằng số	N/A	N/A
Chuẩn	C++/C	C++/C	C++/C	C++/C	C++11 trở lên
Ví dụ	struct Student {...};	union Data {...};	enum Day {...};	typedef int MyInt;	using MyInt = int;

7. VÍ DỤ NGẮN GỌN (TỔNG HỢP)

```
#include <iostream>  
using namespace std;  
typedef struct { int x, y; } Point_t; // typedef với struct  
using ull = unsigned long long; // bí danh dùng 'using'  
  
enum Color { RED, GREEN, BLUE }; // enum  
union Number { int i; float f; }; // union
```

■ ĐIỂM CHÍNH CẦN NHỚ

- ★ Dùng struct để nhóm các kiểu dữ liệu khác nhau.
- ★ Dùng union khi chỉ một thành viên được dùng tại một thời điểm.
- ★ Dùng enum để định nghĩa các hằng số có ý nghĩa.

OOP là mô hình lập trình dựa trên khái niệm "Đối tượng" (Objects), có thể chứa dữ liệu và code.

1. LỚP & ĐỐI TƯỢNG (CLASSES & OBJECTS)

- Class là bản thiết kế (blueprint) hoặc khuôn mẫu.
- Object là một thực thể (instance) của class.

Cú pháp:

```
class TênLớp {
    access_specifier:
        data_members;
        member_functions();
};
```

Ví dụ:

```
class Student {
public:
    string name;
    int age;
    void display() {
        cout << name << " " << age;
    }
};
```

Tạo đối tượng:

```
Student s1; // đối tượng của class Student
```

2. HÀM KHỞI TẠO (CONSTRUCTORS)

- Hàm thành viên đặc biệt có tên trùng với tên class.
- Được gọi tự động khi đối tượng được tạo.
- Dùng để khởi tạo đối tượng.

Cú pháp:

```
TênLớp() {
    // code khởi tạo
}
```

Ví dụ:

```
class Student {
public:
    string name;
    int age;
    Student(string n, int a) {
        name = n;
        age = a;
    }
};
```

Sử dụng:

```
Student s1("Nikhil", 20); // gọi constructor
```

3. HÀM HỦY (DESTRUCTORS)

- Hàm thành viên đặc biệt có dấu ngã (~) trước tên class.
- Được gọi tự động khi đối tượng bị hủy.
- Dùng để giải phóng tài nguyên.

Cú pháp:

```
~TênLớp() {
    // code dọn dẹp
}
```

Ví dụ:

```
class Student {
public:
    ~Student() {
        cout << "Destructor called";
    }
};
```

Lưu ý:

- ★ Constructor → gọi khi tạo đối tượng
- ★ Destructor → gọi khi hủy đối tượng

4. TÍNH ĐÓNG GÓI (ENCAPSULATION)

- Gói dữ liệu (biến) và code (hàm) lại với nhau.
- Ẩn dữ liệu và chỉ cho phép truy cập qua hàm thành viên.

Access Specifiers:

Specifier	Ý nghĩa	Truy cập
private	Chỉ truy cập trong class	X bên ngoài
public	Truy cập mọi nơi	✓ bên ngoài
protected	Truy cập trong class và class dẫn xuất	giới hạn

Ví dụ Encapsulation:

```
class Account {
private:
    double balance; // dữ liệu ẩn
public:
    void deposit(double amt) { balance += amt; }
    double getBalance() { return balance; }
};
```

5. VÍ DỤ ĐẦY ĐỦ

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    string owner;
    double balance;
public:
    // Constructor
    BankAccount(string name, double bal) {
        owner = name; balance = bal;
        cout << "Đã tạo tài khoản cho " << owner << endl;
    }
    // Hàm thành viên
    void deposit(double amt) {
        if (amt > 0) balance += amt;
    }
};
```

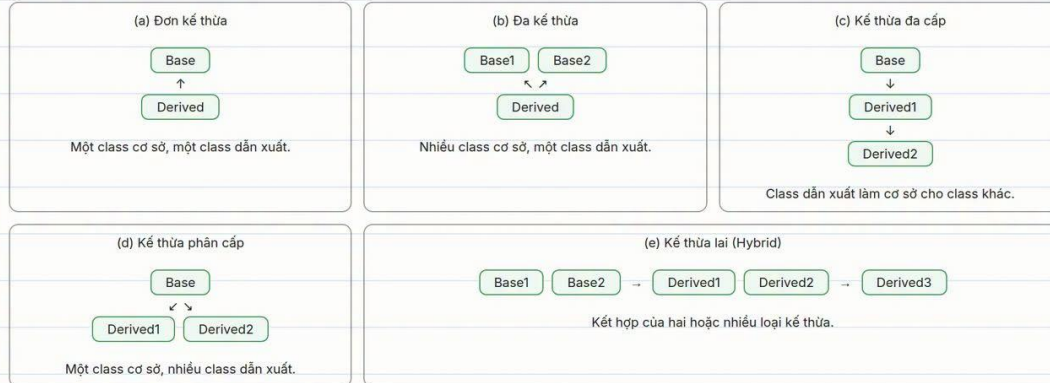
```
void withdraw(double amt) {
    if (amt > 0 && amt <= balance) balance -= amt;
    else cout << "Số dư không đủ!" << endl;
}
double getBalance() { return balance; }
// Destructor
~BankAccount() {
    cout << "Tài khoản của " << owner << " đã đóng." <<
endl;
}
};
```

Output (ví dụ):
Đã tạo tài khoản cho Nikhil
Số dư: 15000
Số dư: 7000
Tài khoản của Nikhil đã đóng.

Kế thừa (Inheritance) cho phép một class mới (dẫn xuất) kế thừa thuộc tính của class có sẵn (cơ sở). Đa hình (Polymorphism) nghĩa là "nhiều dạng". Nó cho phép cùng một interface được dùng cho nhiều dạng khác nhau bên dưới.

1. CÁC LOẠI KẾ THỪ

C++ hỗ trợ các loại kế thừa sau.



2. GHI ĐỀ HÀM (FUNCTION OVERRIDING)

- Khi class dẫn xuất tự định nghĩa lại một hàm đã có trong class cơ sở.
- Đạt được bằng cách dùng cùng chữ ký hàm (function signature).

Ví dụ:

```
class Animal {
public:
    void speak() {
        cout << "Animal speaks" << endl;
    }
};
class Dog : public Animal {
public:
    void speak() { // ghi đè
        cout << "Dog barks" << endl;
    }
};
```

Sử dụng:

```
Dog d;
d.speak(); // Output: Dog barks
```

Lưu ý:

Kiểu trả về phải giống nhau (hoặc covariant).
Access specifier có thể giống hoặc "lỏng" hơn.

3. HÀM ẢO (VIRTUAL FUNCTIONS)

- Khai báo một hàm ở class cơ sở là virtual cho trình biên dịch biết cần resolve lời gọi hàm lúc runtime.
- Cần thiết cho đa hình lúc chạy (runtime polymorphism).

```
class Base {
public:
    virtual void show() {
        cout << "Base show()" << endl;
    }
};
```

Điểm chính

- ★ Hàm ảo có thể được ghi đè.
- ★ Con trỏ/tham chiếu class cơ sở có thể gọi hàm đã ghi đè của class dẫn xuất.
- ★ Constructor không thể là ảo.
- ★ Destructor nên là ảo trong class cơ sở.

4. ĐA HÌNH LÚC CHẠY (RUNTIME POLYMORPHISM)

- Khả năng quyết định gọi hàm nào lúc runtime.
- Đạt được bằng hàm ảo và con trỏ/tham chiếu class cơ sở.

Ví dụ:

```
class Animal {
public:
    virtual void speak() { // ảo
        cout << "Animal speaks" << endl;
    }
};
class Dog : public Animal {
public:
    void speak() override { // ghi đè
        cout << "Dog barks" << endl;
    }
};
int main() {
    Animal* ptr; // con trỏ class cơ sở
    Dog d;
    ptr = &d; // trỏ đến đối tượng dẫn xuất
    ptr->speak(); // resolve lúc runtime
    return 0;
}
```

Luồng xử lý

Kiểu con trỏ: Animal*
Kiểu đối tượng: Dog

Gọi ptr->speak();
↓
Kiểm tra lúc runtime đối tượng đang trỏ tới
↓
Gọi Dog::speak()

Output: Dog barks

5. BẢNG TÓM TẮT

Đặc điểm	Mô tả
Kế thừa	Cơ chế dẫn xuất class mới từ class có sẵn.
Các loại	Đơn, đa, đa cấp, phân cấp, lai.
Ghi đè	Class dẫn xuất định nghĩa lại hàm cùng chữ ký.
Hàm ảo	Hàm class cơ sở khai báo với 'virtual' để resolve lúc chạy.
Đa hình lúc chạy	Lời gọi hàm được resolve lúc runtime bằng hàm ảo.

Trữ tượng hóa (Abstraction) tập trung vào việc chi tiết hóa các đặc điểm thiết yếu và ẩn đi chi tiết triển khai. Các khái niệm OOP nâng cao giúp xây dựng code linh hoạt, dễ bảo trì và an toàn.

1. LỚP TRỮ TƯỢNG (ABSTRACT CLASSES)

- Class chứa ít nhất một hàm ảo thuần túy (pure virtual).
- Không thể tạo đối tượng của class trữ tượng.

Cú pháp:

```
class Shape {
public:
    virtual void draw() = 0; // ảo thuần túy
    void display() { cout << "Shape class" << endl; }
};
```

Ví dụ:

```
class Shape {
public:
    virtual void draw() = 0;
};

int main() {
    Shape s; // X Lỗi: không thể tạo đối tượng
    return 0;
}
```

💡 **Tại sao?** Vì draw() không có phần thân. Class dẫn xuất phải cung cấp phần triển khai.

2. HÀM ẢO THUẦN TÚY (PURE VIRTUAL)

- Được khai báo ở class cơ sở bằng = 0.
- Phải được ghi đè ở class dẫn xuất.

Cú pháp:

```
virtual kiểu_trả_về tên_hàm(tham_số) = 0;
```

Ví dụ:

```
class Shape {
public:
    virtual void draw() = 0; // hàm ảo thuần túy
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }
};
```

Điểm chính

- ★ Làm cho class cơ sở trở thành trữ tượng.
- ★ Buộc class dẫn xuất cung cấp triển khai.
- ★ Hỗ trợ đa hình lúc chạy.

3. INTERFACE TRONG C++ (DÙNG ABSTRACT CLASS)

- C++ không có interface như một số ngôn ngữ khác.
- Ta đạt được interface bằng class trữ tượng chỉ chứa hàm ảo thuần túy.

Ví dụ:

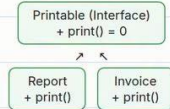
```
class Printable { // Interface
public:
    virtual void print() = 0;
};

class Report : public Printable {
public:
    void print() override { cout << "Printing Report" << endl; }
};

class Invoice : public Printable {
public:
    void print() override { cout << "Printing Invoice" << endl; }
};
```

Cách hoạt động:

Bất kỳ class nào kế thừa Printable phải triển khai print(). Đây giống như một hợp đồng interface.



4. HÀM BẠN (FRIEND FUNCTIONS)

- Không phải hàm thành viên của class nhưng có thể truy cập các thành viên private, protected.
- Khai báo bằng từ khóa friend bên trong class.

Cú pháp:

```
class TênLớp {
    friend kiểu_trả_về tên_hàm(tham_số);
};
```

Ví dụ:

```
class Box {
private:
    int length, breadth, height;
public:
    Box(int l, int b, int h) : length(l), breadth(b), height(h) {}
    friend int volume(Box b); // khai báo hàm bạn
};

int volume(Box b) { // không phải hàm thành viên
    return b.length * b.breadth * b.height;
}

int main() {
    Box box(2, 3, 4);
    cout << "Volume: " << volume(box) << endl; // 24
    return 0;
}
```

💡 Hàm bạn phá vỡ tính đóng gói một cách có chủ đích cho các nhu cầu cụ thể.

5. SO SÁNH NHANH

Khái niệm	Mục đích	Đặc điểm chính	Tạo đối tượng?	Phải ghi đè?	Trường hợp dùng
Abstract Class	Cung cấp cơ sở có hành vi chưa xác định	Ít nhất 1 hàm ảo thuần túy	Không	Có (với hàm ảo thuần túy)	Bản thiết kế cơ sở cho class dẫn xuất
Pure Virtual Function	Buộc class dẫn xuất triển khai	Khai báo với = 0	N/A	Có	Hành vi bắt buộc ở class dẫn xuất
Interface	Định nghĩa một hợp đồng	Tất cả hàm đều ảo thuần túy (trong C++)	Không	Có	Nhiều class theo cùng một hợp đồng
Friend Function	Truy cập thành viên private/protected	Khai báo với từ khóa friend	N/A	Không	Khi hàm không phải thành viên cần truy cập

- ✓ Class trữ tượng cung cấp bản thiết kế với chi tiết được ẩn đi.
- ✓ Trữ tượng hóa ẩn sự phức tạp và chi tiết triển khai những gì cần thiết.

Templates cho phép lập trình tổng quát (generic programming) - viết một lần, dùng cho mọi kiểu dữ liệu.
Xử lý ngoại lệ (Exception Handling) giúp xử lý lỗi runtime một cách "êm ái" mà không làm crash chương trình.

TEMPLATES

1. FUNCTION TEMPLATES

- Một function template có thể dùng cho mọi kiểu dữ liệu.
- Kiểu dữ liệu được chỉ định lúc gọi hàm.

Cú pháp:

```
template <typename T>
kiểu_trả_về_tên_hàm(tham_số)
{
    // code
}
```

Ví dụ:

```
template <typename T>
T max(T a, T b)
{
    return (a > b) ? a : b;
}

int main() {
    cout << max(10, 20) << endl; // 20
    cout << max(3.5, 2.1) << endl; // 3.5
    cout << max('a', 'z') << endl; // z
    return 0;
}
```

2. CLASS TEMPLATES

- Một class template có thể dùng cho mọi kiểu dữ liệu.
- Kiểu dữ liệu được chỉ định lúc tạo đối tượng.

Cú pháp:

```
template <typename T>
class TênLớp {
    T data;
public:
    // members
};
```

Ví dụ:

```
template <typename T>
class Box {
    T data;
public:
    Box(T d) { data = d; }
    void show() { cout<<"Data: "<<data;}
};

int main() {
    Box<int> b1(10);
    Box<string> b2("Hello");
    b1.show(); // Data: 10
    b2.show(); // Data: Hello
    return 0;
}
```

3. NHIỀU THAM SỐ TEMPLATE

- Template có thể có nhiều tham số kiểu.

Ví dụ:

```
template <typename T, typename U>
U getMax(T a, U b)
{
    return (a > b) ? a : b;
}

int main() {
    cout << getMax<int, double>(10, 20.5); // 20.5
    return 0;
}
```

💡 Templates được resolve (biên dịch) lúc compile-time.

XỬ LÝ NGOẠI LỆ

4. Ý TƯỞNG CƠ BẢN

- Exception tách biệt code xử lý lỗi khỏi code bình thường.

Code bình thường → Xây ra lỗi → Ném ngoại lệ → Xử lý êm ái

5. TRY, CATCH, THROW

Cú pháp:

```
try {
    // code có thể ném ngoại lệ
}
catch (KiểuNgoạiLệ e) {
    // xử lý ngoại lệ
}
catch (...) {
    // xử lý mọi ngoại lệ khác
}
throw; // dùng để ném ngoại lệ
```

Điểm chính

- try: theo dõi code có thể gây lỗi
- throw: ném ra một ngoại lệ
- catch: xử lý ngoại lệ
- Chương trình tiếp tục chạy sau khi xử lý.

Ví dụ:

```
#include <stdexcept> // cho runtime_error
int divide(int a, int b)
{
    if (b == 0)
        throw runtime_error("Chia cho 0!");
    return a / b;
}

int main() {
    int x = 10, y = 0, result;
    try {
        result = divide(x, y);
        cout << "Result: " << result;
    }
    catch (runtime_error e) {
        cout << "Error: " << e.what() << endl;
    }
    cout << "Chương trình vẫn tiếp tục..." << endl;
    return 0;
}
```

⚠️ e.what() trả về nội dung thông báo lỗi.

6. NÉM NGOẠI LỆ TỰ ĐỊNH NGHĨA

- Có thể ném bất kỳ kiểu nào (int, string, đối tượng class, ...)

Ví dụ:

```
int main() {
    try {
        throw string("Ôa có lỗi xảy ra!");
    }
    catch (string msg) {
        cout << "Bắt được: " << msg << endl;
    }
    return 0;
}
```

🚫 Bạn có thể ném kiểu built-in hoặc kiểu tự định nghĩa.

7. TEMPLATES vs MACROS vs OVERLOADING

Đặc điểm	Templates	Macros	Function Overloading
Type Safety	Có	Không	Có
Debug	Dễ	Khó	Dễ
Compile Time	Kiểm tra lúc biên dịch	Không kiểm tra	Kiểm tra lúc biên dịch
Hiệu năng	Hiệu quả	Có thể kém hiệu quả	Hiệu quả
Scope	Tuân theo scope	Không có scope	Tuân theo scope

STL là thư viện mạnh mẽ trong C++ cung cấp sẵn các lớp container, thuật toán và iterator.

Thành phần của STL

Containers (lưu dữ liệu) | Algorithms (xử lý dữ liệu) | Iterators (duyệt dữ liệu)

Container thường dùng

- Sequence: vector, list, deque, stack, queue, priority_queue
- Associative: set, map
- Unordered: unordered_set, unordered_map

1. VECTOR

Mảng động

- Truy cập ngẫu nhiên $O(1)$.
- Truy cập nhanh, thêm/xóa chậm.
- Lưu phần tử liên tục trong bộ nhớ.

```
vector<int> v = {1, 2, 3};  
v.push_back(4);  
cout << v[0];
```

2. LIST

Danh sách liên kết đôi

- Chèn/xóa hiệu quả.
- Không truy cập ngẫu nhiên.
- Lưu phần tử không liên tục.

```
list<int> l = {1, 2, 3};  
l.push_back(4);
```

3. DEQUE

Hàng đợi hai đầu

- Chèn, xóa ở cả hai đầu.
- Truy cập ngẫu nhiên $O(1)$.

```
deque<int> d = {1, 2, 3};  
d.push_front(0);  
d.push_back(4);
```

4. STACK

LIFO (Vào sau ra trước)

- Push, pop từ đỉnh.
- Không thể duyệt trong STL stack.

```
stack<int> s;  
s.push(1);  
s.pop();
```

5. QUEUE

FIFO (Vào trước ra trước)

- Chèn ở cuối, xóa ở đầu.
- Không thể duyệt trong STL queue.

```
queue<int> q;  
q.push(1);  
q.pop();
```

6. PRIORITY_QUEUE

Hàng đợi ưu tiên (dạng heap)

- Phần tử được lưu trong heap.
- Phần tử đầu là lớn nhất (mặc định).
- Thao tác $O(\log n)$.

```
priority_queue<int> pq;  
pq.push(10);  
pq.push(5);
```

7. SET

Phần tử duy nhất, có sắp xếp

- Lưu các phần tử duy nhất.
- Tự động sắp xếp.
- Thao tác $O(\log n)$.

```
set<int> s = {1, 2, 3};  
s.insert(4);
```

8. MAP

Cặp khóa-giá trị

- Lưu khóa duy nhất kèm giá trị.
- Khóa được sắp xếp.
- Thao tác $O(\log n)$.

```
map<string, int> m;  
m["one"] = 1;
```

★ Các phương thức STL chính

begin(), end() • size(), empty()
push_back(), pop_back() • insert(), erase()
find(), count()

💡 STL giúp lập trình C++ dễ dàng, nhanh và hiệu quả hơn.

🔥 SO SÁNH CÁC CONTAINER

Container	Truy cập	Chèn/Xóa	Thứ tự	Trùng lặp	Trường hợp dùng
vector	$O(1)$	$O(n)$ (giữa)	Giữ nguyên	Cho phép	Truy cập nhanh, mảng
list	$O(n)$	$O(1)$ (bất kỳ)	Giữ nguyên	Cho phép	Chèn/xóa thường xuyên
deque	$O(1)$	$O(1)$ (2 đầu)	Giữ nguyên	Cho phép	Hàng đợi hai đầu
stack	$O(1)$	$O(1)$	LIFO	Cho phép	Undo, đệ quy
queue	$O(1)$	$O(1)$	FIFO	Cho phép	Lập lịch tác vụ
priority_queue	$O(1)$	$O(\log n)$	Không giữ	Cho phép	Lấy phần tử lớn nhất nhanh

Mẹo dùng STL:

- ✓ Dùng vector khi cần truy cập nhanh.
- ✓ Dùng list khi chèn/xóa thường xuyên.
- ✓ Dùng set/map cho dữ liệu duy nhất, có sắp xếp.
- ✓ Dùng priority_queue khi cần phần tử lớn nhất.

Thuật toán STL là các hàm tổng quát hoạt động trên range (mảng, vector, list, ...).
Iterator dùng để trỏ đến các phần tử trong container và duyệt qua chúng.

1. sort()

Sắp xếp các phần tử theo thứ tự tăng dần (mặc định).

Cú pháp:

```
sort(start, end);
```

Ví dụ (vector):

```
vector<int> v = {4,2,5,1};
sort(v.begin(), v.end());
// v = {1,2,4,5}
```

2. find()

Tìm lần xuất hiện đầu tiên của một giá trị trong range.

Cú pháp:

```
find(start, end, value);
```

Ví dụ (vector):

```
vector<int> v = {1,3,5,7};
auto it = find(v.begin(), v.end(), 5);
if (it != v.end())
    cout<<"Tìm thấy tại chỉ số "
    <<(it - v.begin());
else cout<<"Không tìm thấy";
```

3. reverse()

Đảo ngược thứ tự các phần tử trong range.

Cú pháp:

```
reverse(start, end);
```

Ví dụ (vector):

```
vector<int> v = {1,2,3,4};
reverse(v.begin(), v.end());
// v = {4,3,2,1}
```

4. binary_search()

Kiểm tra một giá trị có tồn tại trong range đã sắp xếp không. Trả về true/false.

Cú pháp:

```
binary_search(start, end, value);
```

Ví dụ (vector):

```
vector<int> v = {1,2,3,4,5};
bool found = binary_search(v.begin(), v.end(), 3);
// found = true
```

5. ITERATORS

Iterator hoạt động giống con trỏ. Chúng dùng để truy cập và duyệt qua các phần tử trong container.

Các loại Iterator

- iterator → đọc/ghi
- const_iterator → chỉ đọc
- reverse_iterator → chiều ngược
- const_reverse_iterator → ngược, chỉ đọc

Ví dụ (vector iterator):

```
vector<int> v = {10,20,30,40};
// dùng iterator
for (vector<int>::iterator it = v.begin(); it != v.end(); ++it)
    cout << *it << " "; // 10 20 30 40
// dùng auto (C++ hiện đại)
for (auto it = v.begin(); it != v.end(); ++it)
    *it *= 2; // đổi giá trị các phần tử
for (int x : v)
    cout << x << " "; // 20 40 60 80
```

Lưu ý

- end() không trỏ đến phần tử hợp lệ.
- Không thể dereference iterator end().
- Iterator cung cấp cách truy cập thống nhất cho nhiều loại container.

Ví dụ (list iterator):

```
list<string> lst = {"C++", "STL", "Rocks"};
for (auto it = lst.begin(); it != lst.end(); ++it)
    cout << *it << " "; // C++ STL Rocks
// duyệt ngược
for (auto rit = lst.rbegin(); rit != lst.rend(); ++rit)
    cout << *rit << " "; // Rocks STL C++
```

6. CUSTOM COMPARATORS

Dùng để thay đổi thứ tự sắp xếp mặc định.

Ví dụ 1: Sắp xếp giảm dần

```
vector<int> v = {4,1,7,3};
sort(v.begin(), v.end(), greater<int>());
// v = {7,4,3,1} → greater<int>() là comparator có sẵn
```

Ví dụ 2: Comparator tự định nghĩa (struct)

```
struct Compare {
    bool operator()(int a, int b) const {
        return a % 2 < b % 2; // chẵn trước, lẻ sau
    }
};
vector<int> v = {5,14,3,8,1,10};
sort(v.begin(), v.end(), Compare());
// Kết quả: {14,8,10,5,3,1}
```

Cách hoạt động?

- Compare() được gọi cho mọi cặp phần tử a, b.
- Nếu Compare(a,b) trả về true, 'a' đứng trước 'b' trong thứ tự sắp xếp.

7. THAM KHẢO NHANH CÁC THUẬT TOÁN STL

Thuật toán	Mục đích	Cú pháp	Header	Độ phức tạp (avg)
sort	Sắp xếp phần tử	sort(start, end)	<algorithm>	O(n log n)
find	Tìm phần tử	find(start, end, value)	<algorithm>	O(n)
reverse	Đảo ngược thứ tự	reverse(start, end)	<algorithm>	O(n)

Header quan trọng

- <algorithm> → các thuật toán
- <vector> → vector
- <list> → list
- <deque> → deque

Modern C++ giúp code an toàn hơn, gọn hơn và diễn đạt tốt hơn.
Viết ít hơn. Làm được nhiều hơn. Tránh lỗi.

1. auto

- Tự động suy luận kiểu của biến.
- Hữu ích với kiểu phức tạp và iterator.

Cú pháp:

```
auto biến = biểu_thức;
```

Ví dụ:

```
int x = 10;
auto y = x;      // int
double d = 3.14;
auto z = d;      // double
vector<int> v = {1,2,3};
auto it = v.begin(); // suy luận là vector<int>::iterator
```

2. VÒNG LẶP RANGE-BASED FOR

- Cú pháp đơn giản hơn để duyệt qua container.
- Được giới thiệu từ C++11.
- Chỉ đọc (dùng const auto&) hoặc theo giá trị.

Cú pháp:

```
for (auto item : container) {
    // dùng item
}
```

Ví dụ:

```
vector<int> nums = {1,2,3,4,5};
for (auto x : nums) {
    cout << x << " "; // 1 2 3 4 5
}
for (const auto& x : nums) {
    cout << x << " "; // truy cập chỉ đọc
}
```

Điểm chính — Code gọn hơn • Ít lỗi hơn • Tự động xử lý iterator

3. HÀM LAMBDA

- Hàm không tên có thể tạo trực tiếp inline.
- Rất phù hợp cho thao tác ngắn, callback, thuật toán.
- Được giới thiệu từ C++11.

Cú pháp:

```
[capture](tham_số) -> kiểu_trả_về { thân_hàm };
```

Ví dụ:

```
// Cộng 2 số
auto add = [](int a, int b) -> int {
    return a + b;
};
cout << add(3, 4); // 7

// Với capture
int factor = 5;
auto mul = [factor](int x) { return x*factor; };
cout << mul(10); // 50
```

Capture: [] không lấy gì • [x] lấy x theo giá trị • [&x] lấy x theo tham chiếu • [=] lấy tất cả theo giá trị • [&] lấy tất cả theo tham chiếu

4. CON TRỎ THÔNG MINH (SMART POINTERS)

- Tự động quản lý bộ nhớ dùng RAII.
- Ngăn ngừa rò rỉ bộ nhớ và con trỏ lơ lửng.

Smart Pointer	Công dụng	Sở hữu
unique_ptr<T>	Sở hữu riêng biệt	Không thể copy
shared_ptr<T>	Sở hữu chung	Đếm tham chiếu
weak_ptr<T>	Quan sát không sở hữu	Hoạt động cùng shared_ptr

Ví dụ:

```
auto up = make_unique<int>(42); // sở hữu riêng
auto sp1 = make_shared<string>("Hello");
auto sp2 = sp1; // sở hữu chung
weak_ptr<string> wp = sp1; // không sở hữu
if (auto s = wp.lock()) {
    cout << *s << endl; // truy cập nếu còn hợp lệ
}
```

✓ Dựa trên RAII ✓ An toàn ngoại lệ ✓ Không cần delete thủ công ✓ Quản lý tài nguyên tốt hơn

5. MOVE SEMANTICS

- Chuyển giao tài nguyên thay vì sao chép.
- Cải thiện hiệu năng cho đối tượng lớn.
- Được giới thiệu từ C++11.

Khái niệm chính: rvalue reference (&&), move constructor, move assignment, std::move()

```
#include <utility>
string s1 = "Hello";
string s2 = move(s1); // chuyển nội dung từ s1 sang s2
cout << s1; // s1 rỗng, chưa xác định
cout << s2; // Hello
vector<int> v1 = {1,2,3,4,5};
vector<int> v2 = move(v1); // không copy phần tử
```

Khi nào dùng? Trả về đối tượng lớn từ hàm • Lưu đối tượng lớn trong container • Tránh sao chép tốn kém

6. nullptr

- Ký tự literal con trỏ null trong C++11.
- An toàn kiểu, khác với NULL hay 0.
- Dùng để biểu diễn một con trỏ null.

Cú pháp:

```
T* ptr = nullptr;
```

Ví dụ:

```
int* p = nullptr;
if (p == nullptr) {
    cout << "Con trỏ null";
}
p = nullptr; // xóa con trỏ
```

Vì sao dùng nullptr?

✓ An toàn kiểu ✓ Tránh nhầm lẫn khi overload ✓ Được ưu tiên hơn NULL và 0

7. TÓM TẮT NHANH

Tính năng	Lợi ích
auto	Suy luận kiểu, ít gõ hơn, ít lỗi hơn
Range-based loops	Duyệt container gọn gàng hơn

Modern C++ = An toàn hơn + Gọn hơn + Nhanh hơn ★ Hãy dùng các tính năng hiện đại để viết code tốt hơn.

C++ nâng cao giúp bạn viết ứng dụng mạnh mẽ, hiệu quả và có khả năng mở rộng. Chương này bao gồm các tính năng quan trọng mà mọi lập trình viên C++ nên biết.

1 XỬ LÝ FILE (FILE STREAMS)

- C++ cung cấp thư viện fstream cho thao tác file.
- Ba class chính: ifstream, ofstream, fstream

Đọc (ifstream)

```
#include <fstream>
using namespace std;
ifstream in("data.txt");
string line;
while(getline(in, line)){
    cout<<line<<'\n';
}
in.close();
```

Ghi (ofstream)

```
#include <fstream>
using namespace std;
ofstream out("data.txt");
out<<"Hello C++"<<endl;
out<<123<< ' '<<4.56;
out.close();
```

💡 Luôn kiểm tra file mở thành công: `if(file.is_open())`

2 NAMESPACES

- Namespace giúp tổ chức code và tránh xung đột tên.
- Dùng toán tử phân giải ::

Ví dụ:

```
namespace Math {
    int add(int a, int b) { return a+b; }
}

int main() {
    int x = Math::add(3, 5);
    using namespace Math;
    int y = add(2, 4);
}
```

✓ Tránh ô nhiễm tên toàn cục ✓ Nhóm các hàm/class liên quan ✓ Có thể lồng nhau ✓ Dùng 'using' cẩn thận

3 TIỀN XỬ LÝ (PREPROCESSOR)

- Chỉ thị tiền xử lý bắt đầu bằng #.
- Dùng để định nghĩa macro, include file, biên dịch có điều kiện.

#include <file>	Bao gồm header file
#define MACRO val	Định nghĩa macro
#ifdef / #ifndef	Nếu macro được / không định nghĩa
#endif	Kết thúc điều kiện

```
#include <iostream>
#define PI 3.14159
#ifdef DEBUG
    cout << "Chế độ Debug\n";
#endif
```

4 QUẢN LÝ BỘ NHỚ

- C++ cho toàn quyền kiểm soát bộ nhớ qua new và delete.
- Luôn giải phóng bộ nhớ đã cấp phát.

```
int* p = new int(10); // cấp phát
double* arr = new double[5];
// sử dụng bộ nhớ...
delete p; // giải phóng 1 biến
delete[] arr; // giải phóng mảng
p = nullptr; arr = nullptr;
```

✓ Tránh rò rỉ bộ nhớ ✓ Giải phóng đúng những gì đã cấp phát ✓ Đặt nullptr sau delete ✓ Ưu tiên dùng smart pointer

5 RAII (Resource Acquisition Is Initialization)

- RAII gắn vòng đời tài nguyên với vòng đời đối tượng.
- Tài nguyên được cấp phát trong constructor và giải phóng trong destructor.

Ví dụ: File wrapper dùng RAII

```
class File {
    fstream file;
public:
    File(const string& name, ios::openmode mode) : file(name, mode) {}
    ~File() {
        if (file.is_open()) file.close();
    }
    fstream& get() { return file; }
};
```

★ **Vì sao RAII?** ✓ Tự động dọn dẹp ✓ Không rò rỉ tài nguyên ✓ An toàn ngoại lệ ✓ Code đơn giản, an toàn hơn

6 SMART POINTERS (Modern C++)

- Con trỏ thông minh tự động quản lý bộ nhớ. • Header: <memory>

Smart Pointer	Công dụng	Sở hữu
unique_ptr<T>	Riêng biệt	Có (auto delete)
shared_ptr<T>	Chung	Có (đếm tham chiếu)
weak_ptr<T>	Quan sát không sở hữu	Không

```
#include <memory>
auto up = make_unique<int>(42);
auto sp1 = make_shared<string>("Hi");
weak_ptr<string> wp = sp1;
if (auto sp = wp.lock()) cout<<*sp<<endl;
```

🛡 Không cần delete thủ công ✓ An toàn ngoại lệ ✓ Code rõ ràng hơn

7 ĐA LUỒNG CƠ BẢN (C++11) 🧑‍🤝🧑

- Dùng <thread> để chạy các tác vụ song song.
- join() để chờ luồng hoàn tất.
- Dùng mutex để bảo vệ dữ liệu dùng chung.

Ví dụ: Mutex cơ bản

```
#include <thread>
#include <mutex>
mutex mtx;
int counter = 0;
void increment() {
    for(int i=0; i<1000; i++) {
        lock_guard<mutex> lock(mtx);
        ++counter;
    }
}
```

8 nullptr (C++11)

- nullptr là từ khóa biểu diễn con trỏ null.
- An toàn kiểu, khác với NULL hay 0.
- Dùng để khởi tạo hoặc so sánh con trỏ.

Ví dụ:

```
int* p = nullptr;
if (p == nullptr) {
    cout << "Con trỏ null";
}
p = nullptr; // tốt hơn NULL hay p = 0
```

Ưu điểm ✓ An toàn kiểu ✓ Không nhầm lẫn overload ✓ Hiện đại, được khuyến nghị dùng bởi C++